

Práctica 4. Manejo de Datos

Tratamiento de Datos, Grado en Ciencia de Datos - UV

Carlos Santiago Martinez Torres

2025-04-02

Contents

Introducción y objetivos.	1
Ejercicios	2
Ejercicio 1	2
Información sobre el df de vuelos	2
Ejercicio 2	3
Ejercicio 3	4
Ejercicio 4	4
Ejercicio 5	4
Ejercicio 6	4
Ejercicio 7	5
Ejercicio 8	5
Ejercicio 9	6
Ejercicio 10	7
Entrega de ejercicios	8

Introducción y objetivos.

El objetivo de esta práctica es manejar data frames estructurados (variables en columnas, muestras en filas) con **tidyR** de una manera eficiente y sencilla utilizando la librería **DPLYR**. Esta librería fue desarrollada por Hadley Wickham (el creador de tidyR) con funciones programadas en C++ de manera óptima. Cuando hablamos de manejar data frames nos referimos a realizar operaciones sobre las muestras y sobre las variables tales como:

1. Seleccionar muestras de un data frame en base a algún tipo de criterio.
2. Seleccionar variables que cumplan alguna condición.
3. Ordenar las muestras de un data frame.
4. Unir varios data frames.

5. Crear nuevas variables a partir de las variables existentes.
6. Agrupar muestras que cumplan algún criterio para realizar cálculos estadísticos.

Para llevar a cabo estas operaciones **DPLYR** proporciona una serie de funciones que enumeramos a continuación:

- `select()`. Permite seleccionar un subconjunto de las variables (columnas) del data frame.
- `filter()`. Esta función, como su propio nombre indica, permite filtrar las muestras (filas) de un data frame en base a algún criterio.
- `group_by()`. Esta función crea grupos de muestras a partir de las categorías de una variable.
- `summarise()`. Esta función facilita el cálculo de parámetros estadísticos de las variables de un data frame.
- `arrange()`. Permite reordenar las muestras de un data frame.
- `XXX_join()`. Las funciones de la familia `XXX_join` permiten combinar data frames.
- `rename()`. Esta función permite renombrar variables de un data frame.
- `mutate()`. Esta función permite añadir nuevas variables (columnas) a partir de la transformación de columnas ya existentes.
- `%>%` Es el operador *pipe* que permite concatenar funciones.

En clase de teoría te han explicado cómo utilizar estas funciones y ya has trabajado con ellas, así que vamos a ver como te desenvuelves con **DPLYR**.

Ejercicios

Vamos a cargar el conjunto de datos `VuelosNY.Rdata` que contiene dos data frames: `flights` y `planes`.

Ejercicio 1

Muestra las primeras muestras de ambos data frames para hacerte una idea de las variables que contienen. ¿Qué diferencia existe entre la información mostrada en ambos data frames?

Información sobre el df de vuelos

```
head(flights)
```

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight
1	2013	1	1	517	515	2	830	819	11	UA	1545
2	2013	1	1	533	529	4	850	830	20	UA	1714
3	2013	1	1	542	540	2	923	850	33	AA	1141
4	2013	1	1	544	545	-1	1004	1022	-18	B6	725
5	2013	1	1	554	600	-6	812	837	-25	DL	461
6	2013	1	1	554	558	-4	740	728	12	UA	1696

	tailnum	origin	dest	air_time	distance	hour	minute	time_hour
1	N14228	EWB	IAH	227	1400	5	15	1357034400
2	N24211	LGA	IAH	227	1416	5	29	1357034400
3	N619AA	JFK	MIA	160	1089	5	40	1357034400
4	N804JB	JFK	BQN	183	1576	5	45	1357034400
5	N668DN	LGA	ATL	116	762	6	0	1357038000
6	N39463	EWB	ORD	150	719	5	58	1357034400

En el df tenemos información sobre la fecha en que se realizó el vuelo, los tiempos de vuelo, distancia, entre otros; es información general sobre el vuelo.

```
head(planes)
```

	tailnum	year	type	manufacturer	model	engines	seats	speed	engine
1	N10156	2004	Fixed wing multi engine	EMBRAER	EMB-145XR	2	55	NA	Turbo-fan
2	N102UW	1998	Fixed wing multi engine	AIRBUS INDUSTRIE	A320-214	2	182	NA	Turbo-fan
3	N103US	1999	Fixed wing multi engine	AIRBUS INDUSTRIE	A320-214	2	182	NA	Turbo-fan
4	N104UW	1999	Fixed wing multi engine	AIRBUS INDUSTRIE	A320-214	2	182	NA	Turbo-fan
5	N10575	2002	Fixed wing multi engine	EMBRAER	EMB-145LR	2	55	NA	Turbo-fan
6	N105UW	1999	Fixed wing multi engine	AIRBUS INDUSTRIE	A320-214	2	182	NA	Turbo-fan

En el df de planes, tenemos información sobre los aviones. El tipo, modelo, numero de motores, etc.

Ejercicio 2

En nuestro estudio queremos aplicar una serie de criterios de exclusión sobre las muestras de ambos data frames. En concreto queremos eliminar los aviones fabricados por el fabricante MCDONNELL DOUGLAS (vigila si existe más de una denominación de esta compañía en la variable `manufacturer`). Del mismo modo, queremos seleccionar aquellos aviones en los que el número de asientos esté entre 90 y 210 (variable `seats`). Aplica las dos operaciones de filtrado secuencialmente utilizando el operador *pipe* y almacena el data frame resultante en `planes.f`. En el data frame resultante unifica las denominaciones del fabricante AIRBUS bajo la denominación "AIRBUS COMP".

- Nota: Revisar el uso de la función `filter` vista en clase y tener en cuenta los operadores de comparación: igual `==`, distinto `!=`, mayor `>`, mayor o igual `>=`, menor `<`, menor o igual `<=`.

```
vector_douglas <- planes %>%
  filter(grepl(c('DOUGLAS', 'MCDONNELL'), manufacturer, ignore.case = TRUE)) %>%
  distinct(manufacturer) %>%
  as.vector()
vector_douglas
```

```
$manufacturer
[1] "DOUGLAS"                "MCDONNELL DOUGLAS"          "MCDONNELL DOUGLAS CORPORATION"
[4] "MCDONNELL DOUGLAS AIRCRAFT CO"
```

```
planes.f <- planes %>%
  filter(!manufacturer %in% vector_douglas) %>%
  filter(seats>=90 & seats <=120)

planes.f$manufacturer[planes.f$manufacturer == 'AIRBUS'] <- 'AIRBUS COMP'

planes.f %>% filter(manufacturer=='AIRBUS') # Comprobamos que se hizo el cambio correcto
```

```
[1] tailnum      year      type      manufacturer model      engines      seats
[8] speed        engine
<0 rows> (o 0- extensión row.names)
```

Ejercicio 3

Crea un data frame denominado `flights.f` que contenga los vuelos que involucren a los aviones presentes en el data frame `planes.f`. Para ello utiliza algunas de las funciones de **XXX_join** y la variable `tailnum` que es común en ambos data frames.

```
flights.f <- flights %>% semi_join(planes.f, 'tailnum') # Semi, solo para quedar con los valores de flig
#flights.f <- semi_join(flights, planes.f, 'tailnum')
```

Ejercicio 4

Reasigna los nombres de los data frames `planes.f` y `flights.f` a los nombres originales `planes` y `flights`.

```
planes <- planes.f
flights <- flights.f
```

Ejercicio 5

Existen variables que, a priori, no tienen demasiado interés para el estudio que queremos llevar a cabo, en particular queremos eliminar:

- En el data frame `flights`, quédate con todas las variables excepto `time_hour` y las variables que terminan con la palabra “time”.
- En el data frame `planes`, quédate con todas las variables excepto `speed` y `type`.

```
flights <- flights %>%
  select(-time_hour) %>%
  select(-ends_with('time'))
colnames(flights) # Verificamos que no esté 'time_hour' y variables que terminene en 'time'.
```

```
[1] "year"      "month"     "day"       "dep_delay" "arr_delay" "carrier"   "flight"   "tailnum"
[9] "origin"    "dest"      "distance"  "hour"      "minute"
```

```
planes <- planes %>% select(-speed, -type)
colnames(planes) # Verificamos que no estén las variables.
```

```
[1] "tailnum"      "year"       "manufacturer" "model"      "engines"    "seats"
[7] "engine"
```

Ejercicio 6

Incorpora una nueva variable a `flights` denominado `Full_Date` que sea de tipo `character` que incorpore una cadena de texto de la forma `d-m-Y`, donde `d` indica día, `m` mes e `Y` año, a partir de la información incluida en el data frame. Utiliza una función de **DPLYR** que permita conseguir este propósito. Elimina las variables originales involucradas en la creación de la nueva variable.

```
flights <- flights %>%
  mutate(Full_Date = paste(day, month, year, sep='-')) %>% # Una vez que crea la nueva variab
  select(-day, -month, -year) # Quitamos los datos que no queremos en el df
head(flights)
```

	dep_delay	arr_delay	carrier	flight	tailnum	origin	dest	distance	hour	minute	Full_Date
1	-3	10	FL	850	N978AT	LGA	MKE	738	7	20	1-1-2013
2	0	11	9E	3538	N915XJ	JFK	MSP	1029	8	10	1-1-2013
3	4	17	FL	346	N977AT	LGA	ATL	762	8	10	1-1-2013
4	-2	2	FL	347	N942AT	LGA	ATL	762	10	23	1-1-2013
5	-1	11	EV	4876	N695CA	EWB	ATL	746	11	45	1-1-2013
6	-8	8	FL	353	N932AT	LGA	CAK	397	11	55	1-1-2013

Ejercicio 7

Utilizando las funciones `group_by`, `summarise` y `filter`, crea un data frame donde se calcule el número de vuelos (`Num_flights`), la distancia media recorrida (`Mean_distance`) y la distancia total recorrida (`Total_distance`) para cada uno de los aviones (`tailnum`) presentes en el conjunto `flights`. Descarta aquellos registros del data frame en los que la variable `tailnum` tenga un valor NA. Utiliza el operador pipe para enlazar las funciones. Almacena el data frame en una variable denominada `Distances_plane`.

```
Distances_plane <- flights %>%
  filter(!is.na(tailnum)) %>%
  group_by(tailnum) %>%
  summarise(Num_flights = n(), Mean_distance = mean(distance), Total_distance = sum(d
head(Distances_plane)
```

```
# A tibble: 6 x 4
  tailnum Num_flights Mean_distance Total_distance
  <chr>      <int>         <dbl>         <dbl>
1 N131EV         55          730.          40144
2 N132EV         35          655.          22932
3 N133EV         38          668.          25394
4 N134EV         38          690.          26229
5 N135EV         42          624.          26220
6 N136EV         37          686.          25400
```

Ejercicio 8

Consigue un data frame denominado `Distance_Manufacturer_model` que contenga 4 columnas (`tailnum`, `Total_distance`, `manufacturer` y `model`) ordenado de mayor a menor en base a la variable `Total_distance`. Para ello une los data frames `Distances_plane` (calculado en el ejercicio anterior y que contiene las distancias totales) y `planes` (que contiene los datos de cada modelo de avión). La variable que te permitirá unir ambos data frames es `tailnum`. A continuación, deberás ordenar, en orden descendente de la variable `Total_distance` el data frame resultante. Finalmente, selecciona las variables que tiene que contener la tabla final (`Total_distance`, `manufacturer` y `model`). Utiliza el operador pipe para enlazar las funciones.

```
Distance_Manufacturer_model <- Distances_plane %>%
  inner_join(planes, by = 'tailnum') %>%
  select(Total_distance, manufacturer, model) %>%
  arrange(desc(Total_distance))
head(Distance_Manufacturer_model)
```

```
# A tibble: 6 x 3
  Total_distance manufacturer  model
  <dbl> <chr>      <chr>
1 40144  N131EV     B737
2 22932  N132EV     B737
3 25394  N133EV     B737
4 26229  N134EV     B737
5 26220  N135EV     B737
6 25400  N136EV     B737
```

```

1      188277 BOMBARDIER INC CL-600-2D24
2      172760 BOMBARDIER INC CL-600-2D24
3      171592 BOMBARDIER INC CL-600-2D24
4      171052 BOMBARDIER INC CL-600-2D24
5      169845 BOMBARDIER INC CL-600-2D24
6      169466 BOMBARDIER INC CL-600-2D24

```

Ejercicio 9

Queremos averiguar si existe algún tipo de relación entre el retraso de los vuelos (llegada y salida) y el fabricante. Para ello queremos calcular el retraso medio en la llegada y en la salida de los vuelos (cuando se disponga de esos datos) para cada uno de los fabricantes (`manufacturer`). Propón la estrategia (y el código) a emplear para conseguir proporcionar un data frame ordenado del retraso en la llegada y la salida para cada uno de los fabricantes. ¿Cuáles son los dos fabricantes de aviones que más se retrasan en las salidas? ¿Y en las llegadas?

```

Delays <- flights %>%
  inner_join(planes, by = 'tailnum') %>% # Para poder ver el fabricante
  filter(!is.na(dep_delay) & !is.na(arr_delay)) %>% # Los retrasos, para cuando no son nu
  select(manufacturer, dep_delay, arr_delay) %>%
  group_by(manufacturer) %>%
  summarise(Mean_DepDelay = mean(dep_delay), Mean_ArrDelay = mean(arr_delay))

Delays

```

```

# A tibble: 4 x 3
  manufacturer Mean_DepDelay Mean_ArrDelay
  <chr>         <dbl>         <dbl>
1 AIRBUS COMP      22.3          11.2
2 BOEING           17.5          18.4
3 BOMBARDIER INC   18.6           8.04
4 DOUGLAS          5.95           3.5

```

```

# ¿Cuáles son los tres fabricantes de aviones que más se retrasan en las salidas?
Delays %>% arrange(desc(Mean_DepDelay)) %>% head(3)

```

```

# A tibble: 3 x 3
  manufacturer Mean_DepDelay Mean_ArrDelay
  <chr>         <dbl>         <dbl>
1 AIRBUS COMP      22.3          11.2
2 BOMBARDIER INC   18.6           8.04
3 BOEING           17.5          18.4

```

```

# ¿Y en las llegadas?
Delays %>% arrange(desc(Mean_ArrDelay)) %>% head(3)

```

```

# A tibble: 3 x 3
  manufacturer Mean_DepDelay Mean_ArrDelay
  <chr>         <dbl>         <dbl>
1 BOEING           17.5          18.4
2 AIRBUS COMP      22.3          11.2
3 BOMBARDIER INC   18.6           8.04

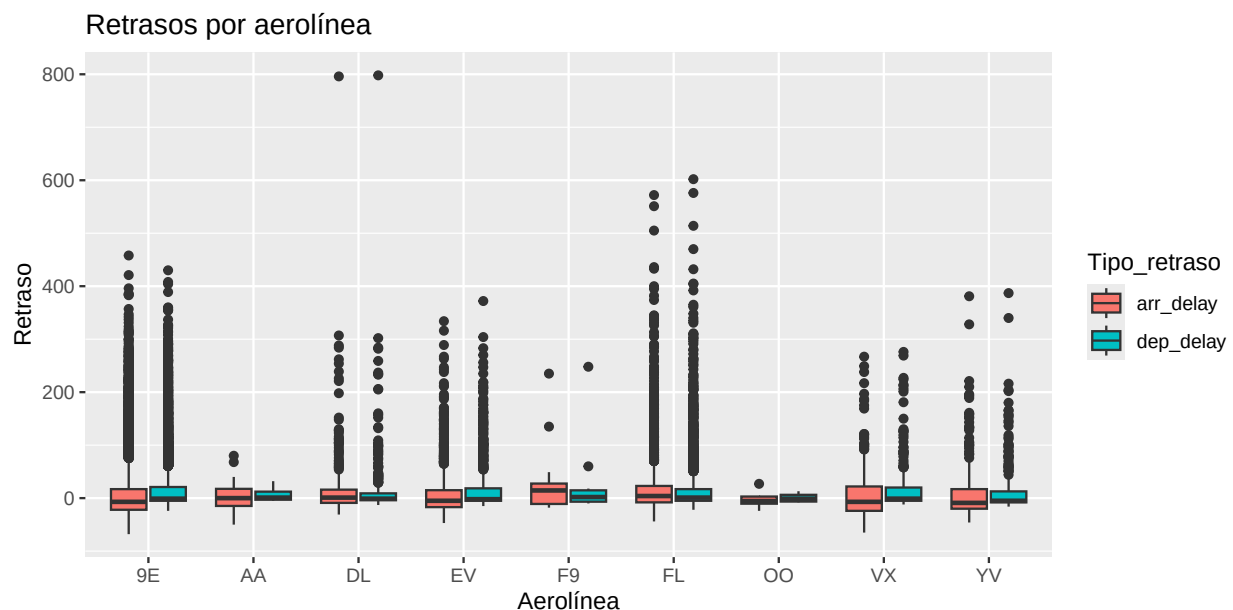
```

Ejercicio 10

Lo que hemos hecho en el ejercicio anterior está bien, pero vale más una imagen que mil palabras. Vamos a plantear representaciones gráficas para visualizar el retraso de cada compañía.

- Representa los boxplots de las variables relacionadas con el retraso en la llegada y en la salida (las que terminan con la palabra `delay`) agrupadas por compañía aérea (variable `carrier`) dibujando en dos colores diferentes cada tipo de retraso: tendrás que modificar el data frame con la función `pivot_longer` para crear una nueva variable que indique el tipo de retraso (llegada y salida) que sirva para colorear agrupando por cada tipo de retraso.

```
flights %>%
  select(ends_with('delay'), carrier) %>%
  pivot_longer(names_to = 'Tipo_retraso', values_to = 'Values', cols = ends_with('delay')) %>%
  ggplot(aes(x=carrier, y=Values, fill=Tipo_retraso)) +
  geom_boxplot() +
  labs (title = 'Retrasos por aerolínea',
        x = 'Aerolínea',
        y = 'Retraso')
```



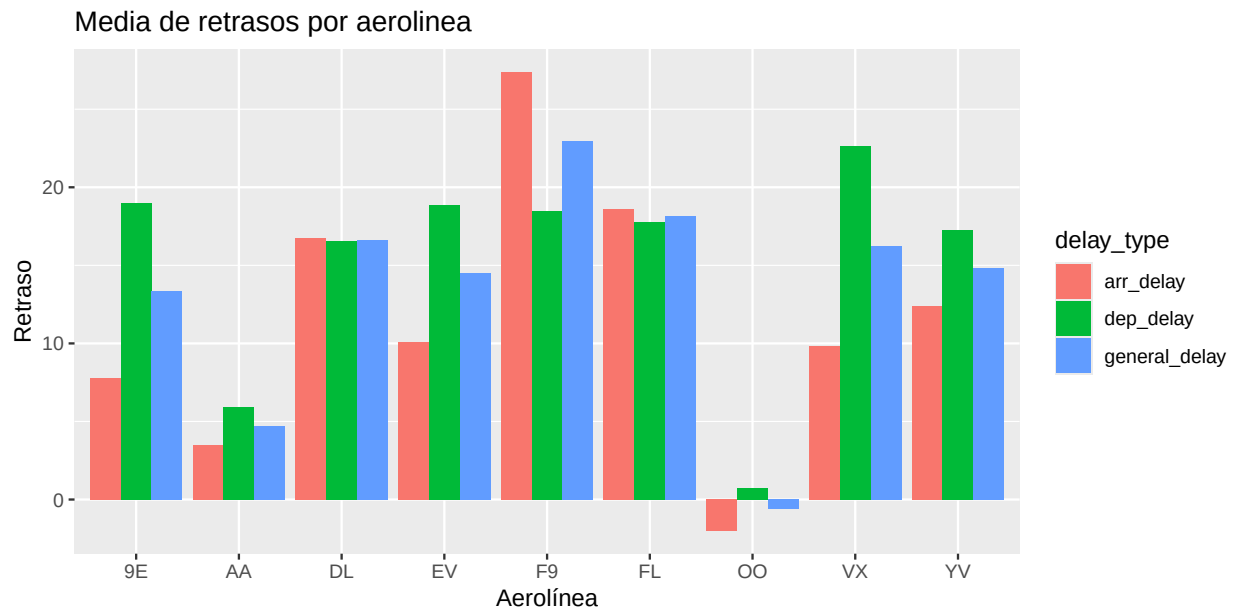
- Representa un gráfico de barras que contenga, para cada compañía aérea (`carrier`), el retraso medio de llegadas, de salidas y el retraso global (nueva variable con el promedio del retraso de salida y el retraso de llegadas). El eje x representará la compañía aérea y el eje y el retraso. Las barras de cada tipo de retraso (llegadas, salidas y global) tendrán un color de relleno diferente (tendrás que modificar el data frame con la función `pivot_longer` para crear una nueva variable que indique el tipo de retraso) y se posicionarán una junto a la otra (no apiladas). No olvides añadir etiquetas en los ejes x e y .

```
flights %>%
  group_by(carrier) %>%
  summarise(arr_delay = mean(arr_delay, na.rm = TRUE), dep_delay = mean(dep_delay, na.rm = TRUE)) %>%
  mutate(global_delay = (arr_delay + dep_delay) / 2) %>%
  pivot_longer(names_to = "delay_type",
```

```

    values_to = "delay",
    cols = c(arr_delay, dep_delay, general_delay)) %>%
ggplot(aes(x = carrier, y = delay, fill = delay_type)) +
geom_col(position = 'dodge') + # Para que no sobreponga las columnas
labs(title = 'Media de retrasos por aerolínea',
     x = 'Aerolínea',
     y = 'Retraso')

```



¿Cuál de las dos representaciones anteriores es más adecuada para visualizar los retrasos?

La representación 2 es más adecuada para visualizar los retrasos, ya que permite comparar los valores entre compañías.

Entrega de ejercicios

Al finalizar la práctica debes completar el cuestionario de aula virtual. Debes subir a aula virtual el fichero RMarkdown compilado en html o pdf con las soluciones a los ejercicios planteados.